

Informe final — Proyecto Scrum

Curso: Scrum y Metodologías Ágiles — Campuslands

Trainer: Duvan Sanabria

Equipo: Juan Polanco, Marlon Sanabria, Michael Martinez

Sprint 1: 14 al 18 de septiembre de 2026

Qué hicimos

Construimos un MVP por consola en Python para gestionar el préstamo de equipos tecnológicos de una institución. La idea era resolver algo concreto: hoy el préstamo se lleva a mano y no hay manera de saber rápido qué equipo está prestado, a quién y desde cuándo.

El objetivo del Sprint era poder registrar equipos y estudiantes, prestar, devolver y que todo quedara guardado en archivos JSON. Eso quedó funcionando.

Planificamos 29 Story Points repartidos en ocho historias: 16 de compromiso firme (HU01 a HU05) y 13 de alcance adicional (HU06 a HU08).

Cómo terminó el Sprint

En la Sprint Review se aceptaron **los 29 puntos**:

- **Aceptadas:** HU01, HU02, HU04, HU05, HU06, HU07 y HU08.
- **Aceptada con observación:** HU03. Funciona y guarda, pero falta pedir el programa académico.

Se cumplió el compromiso firme y también el alcance adicional.

Cómo nos organizamos

Integrante	Rol	De qué se encargó
Juan Polanco	Product Owner	Product Backlog, validar las historias, <code>prestamos.py</code> y <code>main.py</code>
Marlon Sanabria	Scrum Master	Acta de Planning, tablero, <code>estudiantes.py</code> , la función para elegir de una lista y las consultas de préstamos
Michael Martinez	Developer	<code>archivos.py</code> , <code>equipos.py</code> , datos de ejemplo y plan de pruebas

Aunque cada uno tenía un rol, los tres programamos. En un equipo de tres personas no da para que alguien solo administre.

Eventos que hicimos

- **Sprint Planning:** definimos el objetivo, elegimos las historias, las estimamos y las partimos en tareas con responsable.
- **Daily Scrum:** cada uno contó qué había hecho, qué iba a hacer y si algo lo estaba frenando.
- **Sprint Review:** demostramos el incremento y decidí, como PO, qué historias aceptaba.
- **Sprint Retrospective:** hablamos de qué nos funcionó y qué cambiaríamos.

Cada uno tiene su acta y su video.

El código

Está en la carpeta `09_Codigo` y se ejecuta desde ahí con `python main.py`. No usa librerías externas, solo Python.

Quedó separado en módulos, cada uno con una responsabilidad:

- `archivos.py` es el único que lee y escribe los JSON. Así la lectura de archivos está en un solo lugar.
- `equipos.py` maneja el inventario.
- `estudiantes.py` maneja a los estudiantes y sus validaciones.
- `prestamos.py` es el que conecta las dos cosas: presta y recibe devoluciones.
- `main.py` es solo el menú. No tiene lógica, únicamente pide datos y muestra resultados.

Lo que más nos costó pensar fue que el préstamo y el estado del equipo se guarden juntos. Si se guarda uno y no el otro, queda un préstamo registrado con el equipo diciendo que está disponible, y ahí el sistema empieza a mentir.

Pruebas

Las pruebas son manuales, desde la consola. El plan está en `07_Pruebas/Plan_de_Pruebas.md` y tiene 36 casos: por cada historia hay al menos uno positivo y uno negativo, más algunos de borde como opciones inválidas en el menú o un JSON dañado.

Repartimos los casos entre los tres. Se ejecutaron los 36 y **los 36 cumplen**. Cada caso tiene anotado lo que salió en pantalla, y los más importantes tienen su captura.

Las pruebas encontraron dos errores, y los dos se corrigieron:

- **CP-17:** si no había estudiantes registrados, el préstamo dejaba elegir un equipo y solo después avisaba que no había a quién prestárselo. Se cambió `main.py` para revisar primero los estudiantes.
- **Datos de ejemplo:** al reorganizar `estudiantes.json`, los estudiantes quedaron sin su `id` y registrar uno nuevo hacía caer el programa. Se les agregó el `id` de nuevo.

El segundo nos dejó una lección: no basta con probar el código, también hay que volver a probar después de cambiar los datos.

Dificultades

Organizar el trabajo en Git. Lo más difícil no fue programar, fue coordinarnos. Al principio cada uno empezó a subir a un sitio distinto: unos archivos quedaron en la raíz del repositorio y otros dentro de una carpeta. Como los módulos se importan entre ellos, eso hace que el programa no arranque hasta que todos los archivos estén en el mismo lugar. Tuvimos que ponernos de acuerdo en una sola estructura.

Depender del trabajo del otro. Las historias no son independientes. El préstamo necesita que equipos y estudiantes ya existan, y las consultas necesitan que el préstamo funcione. Cuando alguien se atrasaba, el siguiente quedaba bloqueado. Por eso HU04 se programó a mitad de semana y no al final.

Errores pequeños con efectos grandes. Una letra de más en un nombre de variable dejó sin funcionar la lectura de archivos, y con eso todo el programa. Nos sirvió para entender por qué vale la pena probar cada módulo apenas se termina, en vez de esperar a juntarlo todo.

Impedimentos

Durante el Sprint Marlon, como Scrum Master, fue anotando lo que nos frenaba: quién se encargaba de resolverlo y qué hicimos. Tuvimos seis y los seis quedaron resueltos.

Fecha	Impedimento	Responsable	Acción tomada
Lunes 14	No había estructura de carpetas para saber dónde ubicar cada archivo	Marlon	Nadie subió código hasta tener la estructura
Martes 15	Archivos en carpetas distintas; el programa no arrancaba	Michael	Todo el código quedó en <code>09_Codigo</code>
Martes 15	Una letra de más en <code>archivos.py</code> tumbaba todo el programa	Michael	Se corrigió y se acordó probar cada módulo al terminarlo
Jueves 17	HU06 y HU07 esperaban a que HU04 estuviera lista	Juan	Se terminó HU04 primero para desbloquear las consultas
Viernes 18	Los estudiantes de ejemplo quedaron sin <code>id</code> y registrar uno hacía caer el programa	Michael	Se agregó el <code>id</code> y se volvió a probar
Viernes 18	Capturas de pruebas con el mismo nombre se reemplazaron entre sí	Marlon y Michael	Se recuperaron del historial de Git y se nombran por caso

El detalle está en `04_Impedimentos/Registro_de_Impedimentos.md`.

Qué aprendimos

Que Scrum no es solo repartirse tareas. Lo que de verdad ayudó fue tener los criterios de aceptación escritos desde el principio: cuando alguien decía «ya terminé», había una forma objetiva de comprobarlo en vez de discutirlo.

También que estimar en puntos y no en horas tiene sentido. HU06 y HU07 tienen poco código, pero les pusimos 5 puntos porque dependían de otra historia, y esa dependencia fue justamente lo que las dejó

para el final.

Enlaces

Qué	Enlace
Repositorio	https://github.com/Michael-mart7/Proyecto_Scrum
Tablero	https://marlon07721.atlassian.net/jira/software/projects/SCRUM/boards/1
Video de la Sprint Planning	https://drive.google.com/drive/folders/1F3Mm_k4RMjI9YWCgQPWqzSS4b9hFRt3e?usp=sharing
Video de la Daily	https://drive.google.com/drive/folders/1ud2yPLkZvmvlut3YWLTjuKcP2ykn9SCq?usp=sharing
Video de la Sprint Review / Video final	https://drive.google.com/drive/folders/1gx4dXnmH1qaSTQ_N_aUKii9bCHAPOFdv?usp=sharing
Video de la Retrospectiva	https://drive.google.com/drive/folders/1TtaUAodrHXbhbDbJFUrLcil8nfn7RZclV?usp=sharing
Plan de pruebas	https://github.com/Michael-mart7/Proyecto_Scrum/blob/main/07_Pruebas/Plan_de_Pruebas.md
Registro de impedimentos	https://github.com/Michael-mart7/Proyecto_Scrum/blob/main/04_Impedimentos/Rregistro_de_Impedimentos.md
Capturas del tablero	https://github.com/Michael-mart7/Proyecto_Scrum/tree/main/02_Sprint_Planning/Capturas